

Вариант №18

Разработать эмулятор для языка оболочки ОС. Необходимо сделать работу эмулятора как можно более похожей на работу в командной строке UNIX-подобной ОС.

Этап 1. REPL

Цель: создать минимальный прототип. Большинство функций в нем пока представляют собой заглушки, но диалог с пользователем уже поддерживается.

Требования:

1. Приложение должно быть реализовано в форме графического интерфейса (GUI).
2. Заголовок окна должен формироваться на основе реальных данных ОС, в которой выполняется эмулятор. Пример: Эмулятор - [username@hostname].
3. Реализовать простой парсер, который разделяет ввод на команду и аргументы по пробелам.
4. Сообщить об ошибке выполнения команд (неизвестная команда, неверные аргументы).
5. Реализовать команды-заглушки, которые выводят свое имя и аргументы: `ls`, `cd`.
6. Реализовать команду `exit`.
7. Продемонстрировать работу прототипа в интерактивном режиме. Необходимо показать примеры работы всей реализованной функциональности, включая обработку ошибок.
8. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Конфигурация

Цель: сделать эмулятор настраиваемым, то есть поддерживать ввод параметров пользователя в приложение. Организовать для этого этапа отладочный вывод всех заданных параметров при запуске эмулятора.

Требования:

1. Параметры командной строки:
 - Путь к физическому расположению VFS.
 - Путь к стартовому скрипту.

2. Стартовый скрипт для выполнения команд эмулятора: поддерживает комментарии (используйте синтаксис из вашего языка реализации). При выполнении скрипта на экране отображается как ввод, так и вывод, имитируя диалог с пользователем.
3. Сообщить об ошибке во время исполнения стартового скрипта.
4. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования всех поддерживаемых параметров командной строки.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. VFS

Цель: подключить виртуальную файловую систему (VFS).

Требования:

1. Все операции должны производиться в памяти. Запрещается распаковывать или иным образом модифицировать данные VFS, за исключением возможных служебных команд.
2. Источником VFS является директория на диске пользователя.
3. Сообщить об ошибке загрузки VFS (файл не найден, неверный формат).
4. Реализовать команду `vfs-init`, которая заменяет текущую VFS на VFS по умолчанию. В этом случае очищается и физическое представление VFS.
5. Создать несколько скриптов реальной ОС, в которой выполняется эмулятор. Включить в каждый скрипт вызовы эмулятора для тестирования работы с различными вариантами VFS (минимальный, несколько файлов, не менее 3 уровней файлов и папок).
6. Создать стартовый скрипт для тестирования всех реализованных на этом и прошлых этапах команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
7. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Основные команды

Цель: поддержать команды, имитирующие работу в UNIX-подобной командной строке.

Требования:

1. Необходимо реализовать логику для `ls` и `cd`.

2. Реализовать новые команды: `ws`, `cal`, `cat`.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Дополнительные команды

Цель: поддержать более сложные команды, изменяющие состояние VFS, при этом модификации должны осуществляться только в памяти.

Требования:

1. Реализовать команды: `chmod`.
2. По команде `help` выдать список команд с описанием их работы.
3. Создать стартовый скрипт для тестирования всех реализованных на этом этапе команд. Добавить туда примеры всех режимов команд, включая работу с VFS и обработку ошибок.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.